

Complete Beginner Notes: Servers, Data Centers, VMs, Docker & Cloud

1. What is a Server?

A server is simply a computer. The difference is that servers are more powerful than normal laptops/desktops. They usually have:

- More CPU cores
- More RAM
- More storage
- High-speed internet connection
- 24/7 uptime
- Better cooling and power systems

A server's job is to provide services to users or applications.

2. Client vs Server

Client = Requests something.

Server = Responds with something.

Example:

- Browser = Client
- Flask Backend = Server

When you open a website:

Browser → sends request → server processes request → response returns.

3. What is a Data Center?

A data center is a building full of physical servers.

It contains:

- Thousands of servers
- Cooling systems
- Backup electricity
- Networking equipment
- Internet connectivity

Companies like Google, AWS, Netflix, and Meta own huge data centers worldwide.

4. What Happens When You Open a Website?

Step-by-step flow:

1. You type a website URL in browser.
2. Browser sends request through internet.
3. Request travels through routers and ISPs.
4. Nearest data center receives request.
5. Server processes the request.
6. Response comes back to browser.

5. Why Virtual Machines (VMs) Were Created

Physical servers are very powerful.

Example:

- 128 CPU cores
- 512 GB RAM

If only one small app uses the entire server, resources get wasted.

Solution: Divide one physical server into multiple virtual computers. These are called Virtual Machines (VMs).

6. What is a VM?

VM = Virtual Machine.

A VM behaves like a separate computer. Each VM has:

- Its own operating system
- Its own RAM allocation
- Its own storage
- Its own applications

One physical server can run many VMs.

7. What is a Hypervisor?

Hypervisor is software that creates and manages VMs.

Responsibilities:

- Allocate CPU
- Allocate RAM
- Allocate storage
- Manage VM isolation

Hypervisor sits between physical hardware and VMs.

8. What is Cloud Computing?

Cloud computing means renting computing resources over the internet.

Instead of buying physical servers yourself:

- AWS
- Azure
- Google Cloud

provide VMs, storage, networking, and databases on rent.

9. AWS EC2 Understanding

EC2 instance in AWS is usually a Virtual Machine.

When you create an EC2:

- AWS allocates resources from physical server
- Hypervisor creates your VM
- You access it like your own remote computer

10. Why Docker Was Created

VMs are powerful but heavy.

Problem with VMs:

- Each VM has full operating system
- More RAM usage
- Slower startup

Docker solved this by virtualizing applications instead of full computers.

11. What is Docker?

Docker is a platform used to package applications and dependencies into containers.

Docker container includes:

- App code
- Dependencies
- Runtime environment
- Libraries

Containers run consistently everywhere.

12. Docker Image vs Container

Docker Image:

- Blueprint/template
- Static

Docker Container:

- Running instance of image
- Active process

13. What is a Dockerfile?

Dockerfile is a text file containing instructions to build Docker images.

Example responsibilities:

- Install Python
- Install dependencies
- Copy code
- Start application

14. VM vs Docker

VM:

- Full operating system
- Heavy
- Slower

Docker:

- Shares host OS kernel
- Lightweight
- Fast startup

15. Real Modern Infrastructure

Real modern infrastructure often looks like:

Internet → Data Center → Physical Server → Hypervisor → VM → Docker Container → Application

16. Your Fraud Detection Project Infrastructure

Your current project direction:

Frontend → Flutter

Backend → Flask API

ML Model → Fraud prediction model

Containerization → Docker

Deployment → AWS/cloud later

This is becoming a real-world backend + ML engineering project.

Quick Revision Table

Concept	Meaning
---------	---------

Server	A powerful computer
Data Center	Building full of servers
VM	Virtual computer
Hypervisor	Software managing VMs
Cloud	Renting computing resources online
Docker	Container platform
Container	Packaged running application